

PyTorch入門

自分で書けるから、楽をしている

rkskmt

1

前回の振り返り

- NumPy だけで多層ネットワークを組み、初回の図を再現した
- ただし **backward** は **自分で導出して書いた**

❗ クイズ

層の数や活性化関数を変えるたびに、backward を **書き直す** ことになる。100層のネット、1ヶ所も間違えずに書ける？

- 書けるかもしれないが、**人間がやるべき仕事ではない**
- 連鎖律は機械的な作業 — 機械的な作業は機械にやらせる

2

PyTorch とは

- ディープラーニングの **業界標準ライブラリ** のひとつ
- 今日使う機能は実質2つだけ
 - **autograd** : backward を自動でやってくれる
 - **nn** : 層・損失・最適化の部品セット

i ノート

この講義の環境にはインストール済み (**environment.yml** の pytorch-cpu / torchvision)。自分のPCなら **pip install torch** 等に入る。

3

tensor : NumPy 配列の親戚

```
1 import torch
2 import numpy as np
3
4 t = torch.tensor([1.0, 2.0, 3.0])
5 print(t * 2)           # 演算は NumPy とほぼ同じ感覚
6 print(t.numpy())       # NumPy 配列との相互変換も簡単
```

```
tensor([2., 4., 6.])
[1. 2. 3.]
```

- 違いは2つ：**GPU に載せられる** こと、そして**勾配を覚えられる** こと

```
1 a = torch.tensor([0.3, -0.2], requires_grad=True)  # 「勾配を追跡せよ」の印
2 print(a)
```

```
tensor([ 0.3000, -0.2000], requires_grad=True)
```

- **requires_grad=True** を付けた tensor は、**自分が関わった計算をすべて記録** する

山場：autograd が手計算を言い当てる

- バックプロパゲーションの回と **まったく同じ** ネットワーク・同じ数値を流す

```
1 a = torch.tensor([0.3, -0.2], requires_grad=True)  #  $\alpha_1, \alpha_2$ 
2 b = torch.tensor(0.1, requires_grad=True)
3 w = torch.tensor(0.5, requires_grad=True)
4 c = torch.tensor(-0.1, requires_grad=True)
5 x = torch.tensor([1.0, 0.5])  # 入力 (正解 y=1)
6
7 z = a @ x + b  # アフィン変換
8 h = torch.relu(z)  # ReLU
9 u = w * h + c  # 出力層
10 p = torch.sigmoid(u)  # シグモイド
11 L = -torch.log(p)  # 交差エントロピー (正解1)
12
13 L.backward()  # ← 復路はこの1行
14
15 print(f"L = {L.item():.4f}")
16 print(f" $\partial L / \partial \alpha = \{a.grad.numpy().round(4)\}$ ")
17 print(f" $\partial L / \partial b = \{b.grad.item():.4f\}$ ")
18 print(f" $\partial L / \partial w = \{w.grad.item():.4f\}$ ")
19 print(f" $\partial L / \partial c = \{c.grad.item():.4f\}$ ")
```

```
L = 0.6685
 $\partial L / \partial \alpha = [-0.2438 \ -0.1219]$ 
 $\partial L / \partial b = -0.2438$ 
 $\partial L / \partial w = -0.1463$ 
 $\partial L / \partial c = -0.4875$ 
```

① 答え合わせ

手計算（とNumPy検算）の答えは α : $(-0.2438, -0.1219)$ 、 b : -0.2438 、 w : -0.1463 、 c : -0.4875 。 **全桁一致**。 **`loss.backward()`** の中身は、前回自分の手でやった連鎖律の機械化にすぎない。

5

なぜ自動でできるのか

- forward の間、PyTorch は **計算の履歴（計算グラフ）** を記録している
- `backward()`** は、その履歴を **逆順にたどって連鎖律を適用** するだけ
- やっていることは **前回の手計算1往復** と同じ：
 - 往路の途中の値を保存（だからメモリを食う）
 - δ を出力側から掛けて貯める

💡 ヒント

ブラックボックスとして使うのと、中身を知って使うのは別物。 われわれは **中身を手で再現済み** なので、堂々と楽をしてよい。

6

nn：部品には全部、見覚えがある

```
1 import torch.nn as nn
2
3 model = nn.Sequential(
4     nn.Linear(2, 16),    # アフィン変換  $X @ W1 + b1$  (パラメータは自動で用意・初期化)
5     nn.ReLU(),          # 折る
6     nn.Linear(16, 1),    # 出力層  $H @ w2 + c2$ 
7 )
8 print(model)
```

```
Sequential(
  (0): Linear(in_features=2, out_features=16, bias=True)
  (1): ReLU()
  (2): Linear(in_features=16, out_features=1, bias=True)
)
```

PyTorch の名前	この講義で作った部品
<code>nn.Linear</code>	アフィン変換 ($X @ W1 + b1$)
<code>nn.ReLU</code>	$\max(0, z)$
<code>nn.BCEWithLogitsLoss</code>	シグモイド＋交差エントロピー (打ち消しもやってくれる)
<code>torch.optim.SGD</code>	勾配降下法 ($\theta -= lr * grad$)
<code>loss.backward()</code>	バックプロパゲーション

- 新しい概念はゼロ。全部、名前が付いただけ

学習ループ：3拍子

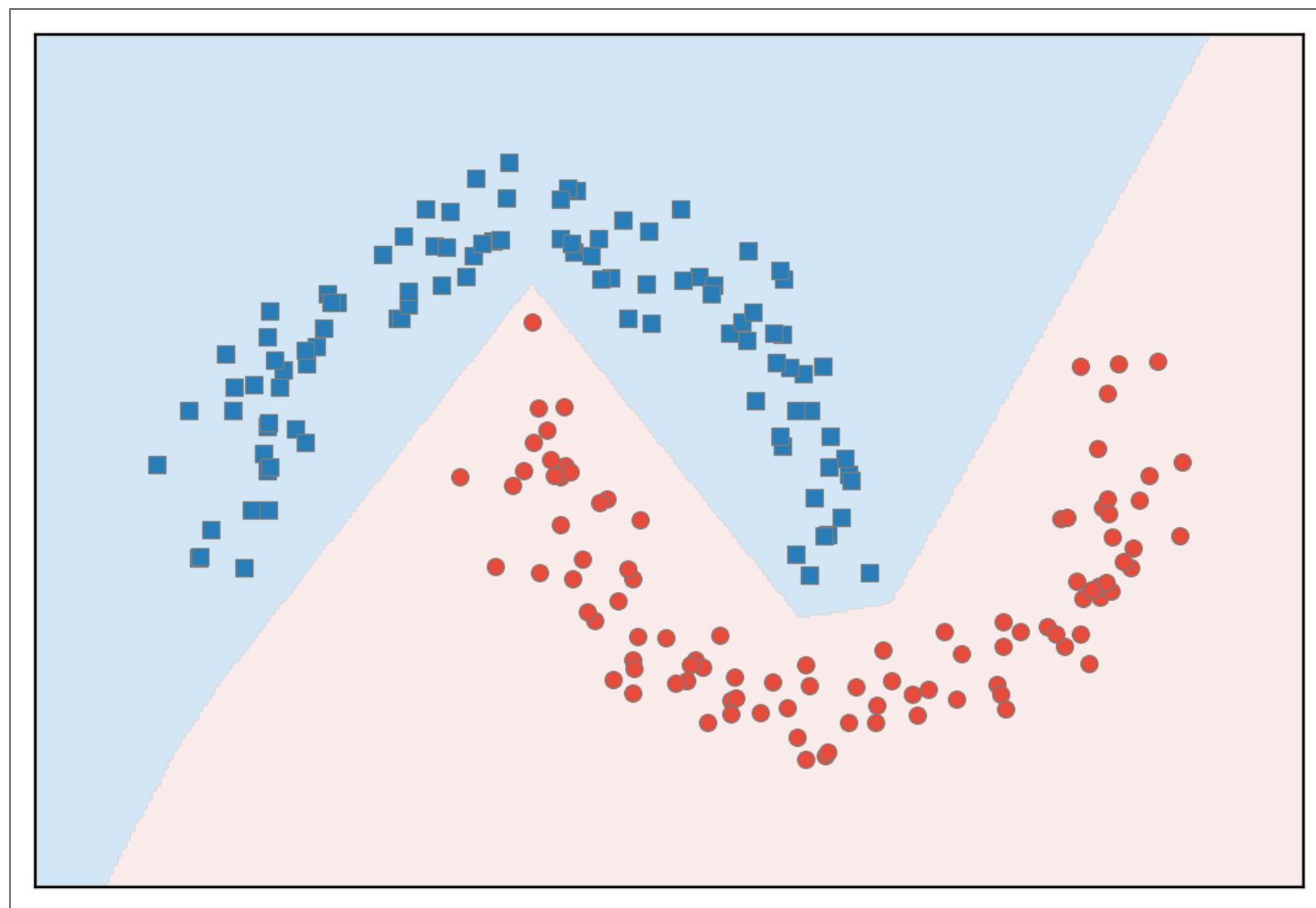
```
1 from sklearn.datasets import make_moons
2
3 X, y = make_moons(n_samples=200, noise=0.1, random_state=42)
4 X_t = torch.tensor(X, dtype=torch.float32)
5 y_t = torch.tensor(y, dtype=torch.float32).unsqueeze(1)  # (200,) → (200,1)
6
7 torch.manual_seed(0)
8 model = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 1))
9 loss_fn = nn.BCEWithLogitsLoss()
10 optimizer = torch.optim.SGD(model.parameters(), lr=0.5)
11
12 for i in range(1, 4001):
13     optimizer.zero_grad()          # ① 前回の勾配をリセット
14     logits = model(X_t)
15     loss = loss_fn(logits, y_t)
16     loss.backward()                # ② backprop (自動)
17     optimizer.step()               # ③ 勾配降下で1歩
18
19     if i in (1, 100, 500, 2000, 4000):
20         acc = ((torch.sigmoid(logits) >= 0.5).float() == y_t).float().mean()
21         print(f"更新 {i:5d}回 | Loss: {loss.item():.4f} | 正答率: {acc.item():.1%}")
```

更新	1回		Loss: 0.6807		正答率: 50.0%
更新	100回		Loss: 0.2652		正答率: 85.5%
更新	500回		Loss: 0.0449		正答率: 99.0%
更新	2000回		Loss: 0.0057		正答率: 100.0%
更新	4000回		Loss: 0.0024		正答率: 100.0%

- NumPy 版と同じく **100%**。 **zero_grad → backward → step** の3拍子はどんなネットでも同じ

境界も同じものが出る

▶ 描画コード



- NumPy 版で書いた forward / backward 計60行ほどが、PyTorch では**学習ループ込みで十数行**
- 浮いた労力は、**モデルとデータを考えること**に使う

9

まとめ

- **autograd** : **loss.backward()** = 連鎖律の自動化。手計算と全桁一致を確認した
- **nn の部品** は全部、この講義で自作した部品の別名（対応表）
- 学習ループは **zero_grad → backward → step** の3拍子
- 中身を知っているから、安心して楽ができる

① 次回予告：最終回

道具は完成した。次回、初回に約束した**手書き数字を読み取る機械**を作る。入力は 28×28 ピクセル — **784次元**。次元の話、覚えてる？

10